

Polimorfismo

Programação Funcional



Polimorfismo

- Aumento da reusabilidade
 - Alta produtividade de software
- Permite aplicar uma mesma função a vários tipos de dados, representados por um tipo variável
- Diferente de sobrecarga (polimorfismo ad hoc)
 - Funções com o mesmo nome aplicadas a vários tipos de dados
- Haskell permite os dois tipos de polimorfismo
- Sobrecarga é feita através de um mecanismo chamado de type class



Polimorfismo

- Exemplo

```
length :: [t] -> Int  
length [] = 0  
length (a:x) = 1 + length x
```

- **t** é um tipo polimórfico porque pode ser aplicado a qualquer tipo de lista homogênea



Polimorfismo

- Dificuldades podem surgir nas definições dos tipos das funções quando elas envolvem tipos variáveis
 - Podem existir muitas instâncias de um tipo variável
- Solução: É necessário que o tipo da função seja o mais geral possível
- Um tipo w de uma função f é "o tipo mais geral" de f se todos os tipos de f forem instâncias de w
- Exemplo:
 - Função zip
 - $[t] \rightarrow [t] \rightarrow [(t, t)]$ (não é o mais geral)
 - $[t] \rightarrow [u] \rightarrow [(t, u)]$ (mais geral)



Exemplo

- Função fazPar que constrói uma lista de pares a partir de duas listas de inteiros

```
fazPar :: [Int] -> [Int] -> [(Int,Int)]  
fazPar n m = [ (a, b) | a <- n, b <- m]
```

```
fazPar :: [t] -> [u] -> [(t,u)]  
fazPar n m = [ (a, b) | a <- n, b <- m]
```



Exemplos

```
rep :: Int -> t -> [t]
rep 0 ch = [ ]
rep n ch = ch : rep (n - 1) ch
```

```
fst :: (t, u) -> t
fst (x, _) = x
```

```
snd :: (t, u) -> u
snd (_, y) = y
```

```
head :: [t] -> t
head (a : _) = a
```

```
tail :: [t] -> [t]
tail (_ : x) = x
```



Exercícios

- Defina uma função `concat` onde `concat [e1, ..., ek] = e1 ++ ... ++ ek`. Qual o tipo de `concat`?
- Defina uma função `unZip` que transforma uma lista de pares em um par de listas. Qual o seu tipo?
- Defina uma função `init::[t]->[t]` que retorna todos os elementos de uma lista com exceção do último
- Defina as funções `tome` e `tire` do tipo `Int->[t]->[t]` onde `tome n l` retorna os `n` primeiros elementos da lista `l` e `tire n l` retira os `n` primeiros elementos da lista `l`

